

When Correctness Feedback Erases Evidence: Phase-Separated Completeness Auditing in Verus

Adarsh Agrawal, Senior Member, IEEE

Seattle, Washington, USA

Abstract

Formal verification establishes that a program satisfies its written contract, but not that the contract captures what its author intended. We study completeness auditing as a search for a concrete program that passes a weakened contract while violating intended behavior. This objective creates a conflict with familiar correctness feedback: verifier messages and visible tests can steer the model away from the wrong program that would expose the missing obligation. Across 12 Verus/Rust tasks, four model families, and three labeled repeat episodes, no-feedback adversarial discovery certifies 85 of 144 outcomes, while INTENTSHIELD, which provides corrective feedback during discovery, certifies 39. The difference favors no-feedback discovery on 11 of 12 tasks and is 31.9 points on average (task-bootstrap 95% interval [21.5, 38.9], $p = 0.0063$). A phase-separated execution of the same no-feedback protocol independently certifies 85 of 144 outcomes before freezing the selected candidate for repair. In a separate 16-task evaluation, phase-revised specifications yield 39 of 48 successful implementations, compared with 30 of 48 for INTENTSHIELD-revised specifications and 14 of 48 for the original weakened specifications. The effect did not reproduce in the available Dafny or Lean evaluations, and the downstream study lacks a Direct-Revised baseline. Within the studied Verus setting, the result supports a simple rule: preserve and certify the adversarial witness before beginning correctness-oriented repair.

Introduction

A verifier can discharge every obligation in a contract while the resulting program still violates its author’s intent. This is not a failure of verification. It is a failure to write the missing obligation down.

A useful audit therefore searches for a program that the contract accepts but the task rejects. We call such a program a *specification-completeness witness*. Unlike an ordinary synthesis failure, the wrong program is valuable evidence: it shows exactly what the contract forgot to forbid.

This changes how feedback should be used. In ordinary synthesis, a failed proof or test tells the model what to repair. During completeness auditing, the same feedback can push the model toward the intended solution before the admitted failure is preserved. We call this conflict the *objective reversal*.

Our primary comparison covers three policies. DIRECT makes independent adversarial attempts without corrective feedback. INTENTSHIELD feeds compilation, verifier, and visible-test results back into discovery. PHASE is a separate execution of the same discovery protocol as DIRECT, followed by a frozen

handoff to certification and repair. The complete campaign also includes a Model-Judgment feedback policy, reported as a secondary result in Appendix C. We then compare implementation synthesis under the original weakened specification and specifications repaired from certified witnesses.

Our contributions are:

- task-level evidence that correctness-oriented feedback substantially lowers certified-witness yield in the studied Verus setting;
- a phase-separated workflow that preserves the witness before repair; and
- a downstream evaluation connecting witness preservation to stronger repaired specifications, together with negative transfer results that bound the claim.

Completeness auditing

Certified witnesses

Let S^- be a contract with one meaningful postcondition removed. A candidate program p is a certified completeness witness when:

1. p verifies under S^- ;
2. restoring the omitted postcondition makes verification fail;
3. p compiles and executes; and
4. p disagrees with at least one official test case.

Each episode produces three candidates. The harness ranks them lexicographically by parse success, weakened-contract acceptance, Rust compilation, visible-test divergence count, and then earliest call index. Only the selected candidate is certified. Selection uses 16 visible oracle cases; certification replays the selected candidate against the restored contract and complete official oracle bank. Because visible cases can satisfy the behavioral-disagreement criterion, the evaluation demonstrates specification incompleteness rather than held-out generalization. Eight mutation examples per task are exposed as search guidance but do not independently certify wrongness.

Separating discovery from repair

Figure 1 shows the central distinction. INTENTSHIELD keeps corrective feedback inside the discovery loop, allowing later calls to revise earlier candidates. PHASE first searches without that feedback, freezes the exact candidate, and only then certifies the witness and repairs the specification.

The phase boundary is the method, not a new search algorithm. Phase-separated discovery used prompts and settings byte-identical to those used by DIRECT, but issued separate API requests. Both executions obtained 85 of 144 certified witnesses. This provides internal repeat-run stability evidence, although

IntentShield: corrective feedback remains inside discovery

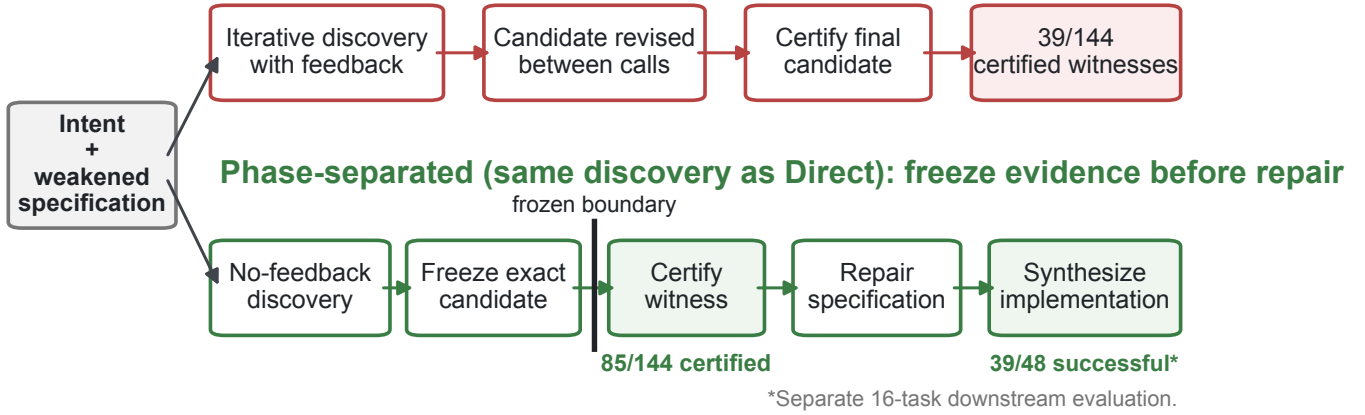


Figure 1: Correctness feedback enters at different times. **INTENTSHIELD** uses it while searching for a witness. **PHASE** uses the same discovery policy as **DIRECT** and introduces feedback only after the candidate has been frozen. The downstream result comes from a separate 16-task evaluation.

their shared harness and selection logic mean implementation-specific artifacts remain possible. The purpose of **PHASE** is to make the handoff from adversarial discovery to ordinary repair explicit and auditable.

Experimental design

Verus tasks

We use 12 functions from VeriContest (Xie et al., 2026) with Verus (Lattuada et al., 2023). For five *residual-contract* tasks, removing one postcondition leaves another meaningful output constraint. For seven *total-omission* controls, the complete output relation is removed while input preconditions remain. Appendix C lists every task and omission.

Candidates are side-effect-free Rust expressions. Every selected candidate is checked under the weakened and restored contracts, compiled as Rust, and evaluated on official cases.

Discovery policies

DIRECT. Three independent calls request a program that satisfies the weakened contract but violates the task. No candidate, verifier result, or visible-test result is carried between calls.

INTENTSHIELD. Each new call receives the previous candidate together with compilation, weakened-contract, and visible-test feedback. The prompt continues to request an admitted wrong program, but the interaction resembles a conventional repair loop.

Model-Judgment. Each new call receives the previous candidate and asks the model to assess whether it verifies under the weakened contract, violates intended behavior, and constitutes a valid witness before revising it. This feedback policy is part of the complete campaign but not the primary inferential comparison.

PHASE. Discovery is identical to **DIRECT**. The chosen candidate is then frozen and certified before any specification repair or correctness-oriented implementation work begins.

Primary matrix

The primary matrix covers 12 tasks, four model families, three labeled repeat episodes, and four executed policies, giving 144 outcomes per policy and 1,728 model calls in total. Primary inference compares **DIRECT**, **INTENTSHIELD**, and **PHASE**; Model-Judgment is secondary. The model families are Claude Sonnet 5, DeepSeek V3.2, Devstral 2, and Qwen3-Coder-480B. Each (task, model, method) combination has three repeat episodes, identified by tracking labels 17, 43, and 101; each episode contains three calls. The labels are not provider seeds. The task is the independent unit, and within-task episodes are repeated stochastic samples rather than independent observations.

All reported model invocations occurred in September 2026 through Amazon Bedrock in us-west-2. Appendix A gives the exact Runtime, inference-profile, and Mantle identifiers. Within each model, policies received matched three-call budgets. Optional sampling controls were omitted, so provider defaults applied. Claude 5 used its default adaptive-thinking configuration and default high effort; GPT-5.6 Terra used the default medium reasoning effort. Runtime open-weight routes used their provider-default decoding parameters. Transient transport failures were retried with the campaign’s adaptive retry policy, while parse failures remained in the denominator. Exact prompt templates with formatting normalized for print appear in Appendix B.

Downstream evaluation

Campaign A2 compares implementation synthesis under the original weakened specification and specifications repaired from certified witnesses. It uses the 12 primary tasks and four additional VeriContest tasks. Each of the following specification conditions receives three direct implementation attempts

per task:

- **Original:** the weakened specification;
- **IS-Revised:** a specification repaired from a `INTENTSHIELD` witness; and
- **Phase-Revised:** a specification repaired from a phase-preserved witness.

The same implementation stage follows each specification condition, and all three attempts contribute to the reported totals. A Direct-Revised condition, in which the specification is strengthened without either witness source, was not evaluated. The downstream study can therefore compare the two witness sources, but cannot isolate witness guidance from specification strengthening itself.

The two-route A2 evaluation used Claude Sonnet 5 and GPT-5.6 Terra in September 2026, with identifiers and default generation settings listed in Appendix A. Appendix B gives formatting-normalized specification-repair and implementation-synthesis templates.

Analysis

For the primary result, each task contributes its `DIRECT`-minus-`INTENTSHIELD` difference over 12 model-run outcomes. We report the mean task effect, a task-cluster bootstrap interval, and an exact two-sided sign test. Downstream comparisons use the difference in successful attempts within each task; ties are omitted from the sign test. The supplement includes the task-level tables and scripts used for every reported statistic and figure.

Results

Corrective feedback suppresses witnesses

`DIRECT` certifies 85 of 144 outcomes (59.0%), compared with 39 of 144 (27.1%) for `INTENTSHIELD`. The mean task-level difference is 31.9 percentage points, with a task-bootstrap 95% interval of [21.5, 38.9]. Eleven tasks favor `DIRECT` and one favors `INTENTSHIELD`, giving an exact two-sided sign-test $p = 0.00635$.

The result is present in both task strata. The difference is 38.3 points on the five residual-contract tasks and 27.4 points on the seven total-omission controls. The only reversal is `cf50A`, where `INTENTSHIELD` certifies four outcomes and `DIRECT` certifies two.

Phase separation preserves discovery

`PHASE` also certifies 85 of 144 outcomes and matches `DIRECT` on every task. This does not mean that `PHASE` searches better. A separate execution yielded the same aggregate rate, providing internal repeat-run stability evidence for the no-feedback protocol. Shared harness and selection logic remain common sources of possible artifact.

Downstream synthesis under repaired specifications

In the Original condition, 42 of 48 implementations verify against the weakened contract, but only 14 verify after the missing postcondition is restored. This 58.3-point gap shows why verification against an incomplete contract is not enough

to establish intended behavior.

Phase-Revised specifications produced 39 of 48 successful implementations (81.3%), compared with 30 of 48 (62.5%) for IS-Revised and 14 of 48 (29.2%) for Original. Phase-Revised exceeds IS-Revised on nine tasks and ties on seven, with no losses ($p = 0.003906$, two-sided sign test); the mean difference is 18.8 points ([10.4, 27.1]). Phase-Revised also exceeds Original on 13 tasks and ties on three ($p = 0.000244$, two-sided sign test). Without Direct-Revised, this result should be read as evidence about the two evaluated witness sources, not as proof that witness-guided repair beats every reasonable strengthening method.

The effect did not reproduce in transfer evaluations

In the available Dafny experiments, frontier routes were at ceiling and the Qwen stratum with limited headroom slightly favored `INTENTSHIELD` over `DIRECT` (23/24 versus 21/24). On 24 Lean tasks from Verina (Ye et al., 2025), `PHASE` is the lowest arm: it solves 8 of 48 cells, compared with 12 for `DIRECT` and 14 for both `INTENTSHIELD` and a no-context control. Appendix E shows the complete transfer summary.

Discussion and limitations

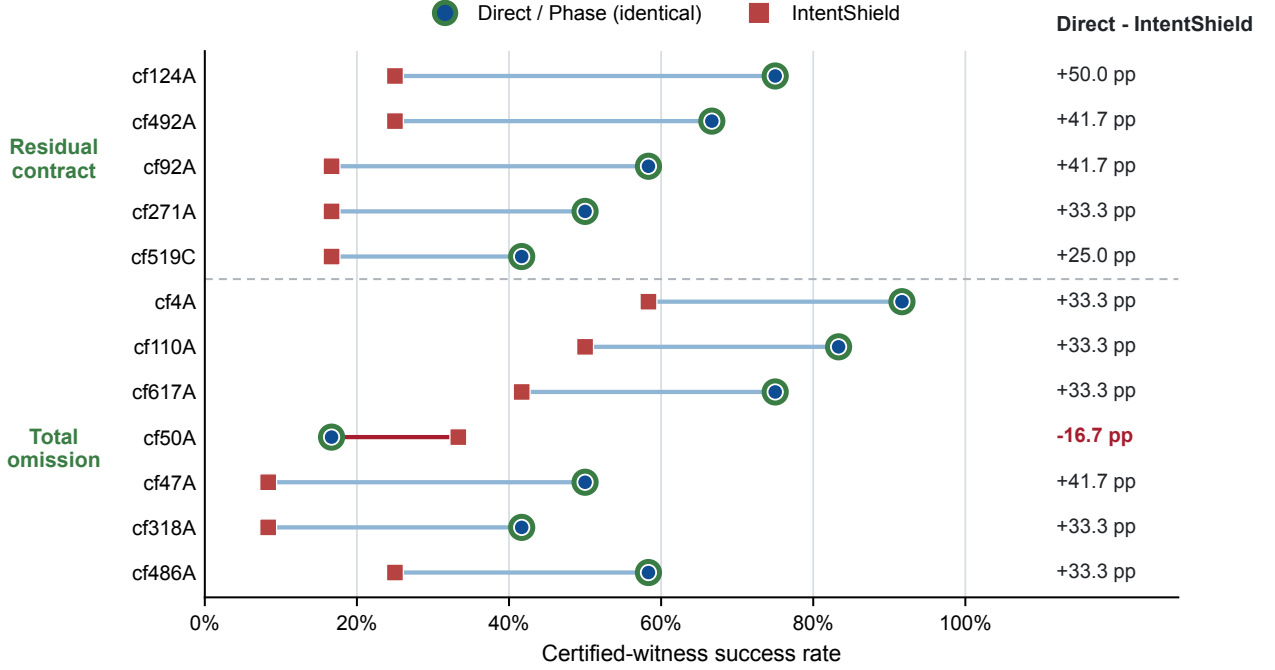
What the result establishes. In the studied Verus setting, the complete `INTENTSHIELD` feedback loop underperforms no-feedback discovery. The experiment changes several channels together, so it does not identify one feedback message as the cause. Parse failures are more frequent under `INTENTSHIELD`, but a parse-clean sensitivity analysis still leaves a 23.6-point advantage for `DIRECT`. Smaller format-matched ablations point in the same direction but are underpowered.

Where the result may fail. The primary benchmark contains 12 competitive-programming functions and a restricted expression language. The effect did not reproduce in the available Dafny or Lean evaluations. We therefore limit the finding to the studied Verus/Rust substrate and task class rather than presenting it as a universal property of correctness feedback. We do not claim that it applies to repositories, mutable state, concurrency, or other proof workflows.

Downstream and evidence limits. The missing Direct-Revised baseline prevents a clean comparison with direct specification strengthening. The current anonymous supplement reproduces all reported statistics and figures from task-level data. Full replay material for the remote downstream campaign will be added when the corresponding prompts, responses, checker records, and execution details are supplied.

Potential use. Concrete admitted failures can help developers repair incomplete proof boundaries. The same search could also expose underspecified behavior, so witnesses should be handled as repair evidence rather than deployment-ready exploits.

Corrective feedback lowers witness yield on 11 of 12 tasks



Overall: 59.0% vs 27.1%, difference +31.9 pp; 95% task-bootstrap interval [21.5, 38.9]; exact sign test $p = 0.00635$. $n = 12$ outcomes per method and task.

Figure 2: Certified-witness rates for each Verus task. Each point summarizes 12 model-run outcomes per policy. DIRECT and PHASE are separate executions of the same no-feedback discovery policy. The task is the independent unit.

Related work

Verus and VeriContest study verified Rust generation (Lattuada et al., 2023; Xie et al., 2026); Verina and VerifyThisBench evaluate generation of programs, specifications, and proofs (Ye et al., 2025; Deng et al., 2025). Agentic and recursive systems generally optimize proof completion (Aggarwal et al., 2024; Breen et al., 2026; Tan, 2026). Our objective is different: find a program admitted by an incomplete proof boundary.

Recent work distinguishes faithful intent formalization from verifier acceptance (Lahiri, 2026; Ye et al., 2026; Misu et al., 2026). Conventional counterexample-guided synthesis removes failing candidates (Solar-Lezama et al., 2005; Alur et al., 2013); completeness auditing must preserve the admitted failure long enough to identify the missing obligation. This connects to specification gaming (Amodei et al., 2016; Leike et al., 2017; Nishimura-Gasparian et al., 2026) and complements self-refinement methods that optimize ordinary task performance (Madaan et al., 2023).

Conclusion

Correctness feedback is not always aligned with the current objective. During Verus completeness auditing, it substantially reduces the rate at which models produce certified admitted failures. A separate execution of the no-feedback protocol obtained the same aggregate rate, and Phase-Revised specifications produced higher observed downstream synthesis success than the evaluated INTENTSHIELD-based repair. The finding is limited to the studied setting, but the engineering lesson is

simple: preserve the failure before asking the model to fix it.

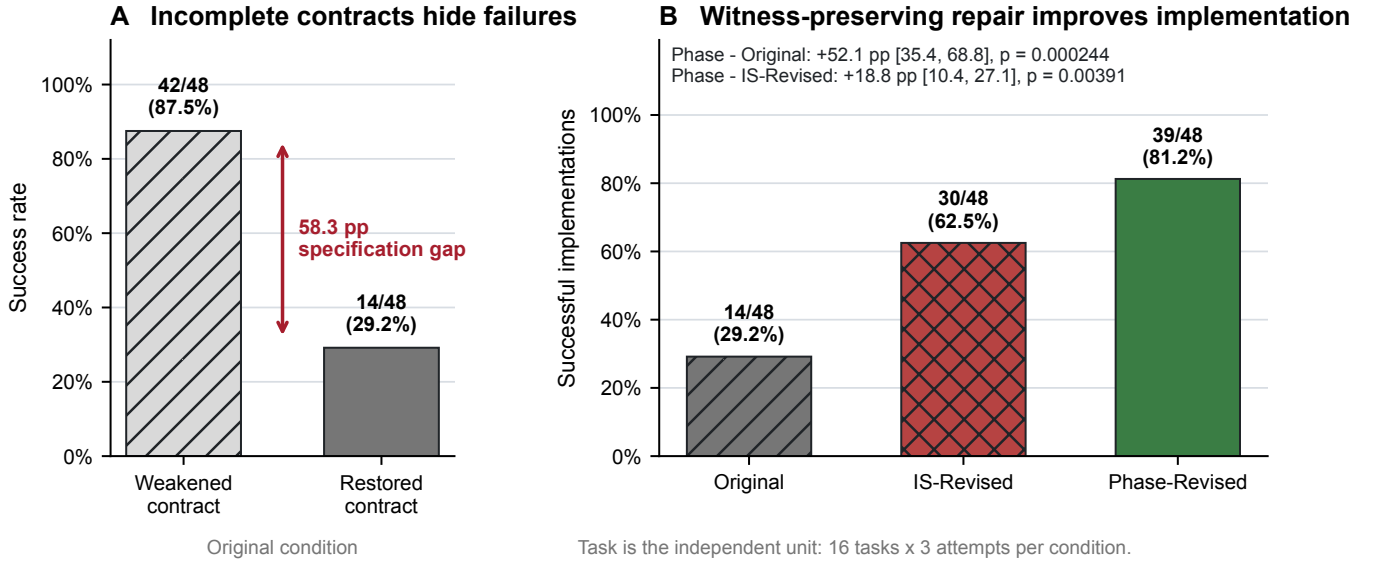


Figure 3: Downstream consequences of incomplete and repaired specifications. Panel A compares weakened-contract verification with restored-contract success in the Original condition. Panel B reports implementation success across the three specification conditions. Intervals resample the 16 task clusters.

References

- Pranjal Aggarwal, Bryan Parno, and Sean Welleck. Alphaverus: Bootstrapping formally verified code generation through self-improving translation and tree refinement. *arXiv preprint arXiv:2412.06176*, 2024.
- Rajeev Alur et al. Syntax-guided synthesis. In *Proceedings of FMCAD*, 2013.
- Dario Amodei et al. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Benjamin Breen, Austin Letson, Borja Requena Pozo, and Leopoldo Sarra. Axdafny: Agentic verified code generation in dafny. *arXiv preprint arXiv:2606.32007*, 2026.
- Xun Deng, Sicheng Zhong, Barış Bayazit, Andreas Veneris, Fan Long, and Xujie Si. Verifythisbench: Generating code, specifications, and proofs all at once. *arXiv preprint arXiv:2505.19271*, 2025.
- Shuvendu K. Lahiri. Intent formalization: A grand challenge for reliable coding in the age of ai agents. *arXiv preprint arXiv:2603.17150*, 2026.
- Andrea Lattuada et al. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023. doi: 10.1145/3586037.
- Jan Leike et al. Ai safety gridworlds. *arXiv preprint arXiv:1711.09883*, 2017.
- Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- Md Rakib Hossain Misu, Iris Ma, and Cristina V. Lopes. Veriact: Beyond verifiability – agentic synthesis of correct and complete formal specifications. *arXiv preprint arXiv:2604.00280*, 2026.
- Kei Nishimura-Gasparian, Robert McCarthy, and David Lindner. Towards understanding specification gaming in reasoning models. *arXiv preprint arXiv:2605.02269*, 2026.
- Armando Solar-Lezama, Rodric Rabbah, Rastislav Bodik, and Kemal Ebcioglu. Programming by sketching for bitstreaming programs. In *Proceedings of PLDI*, 2005.
- Max Tan. Automating formal verification with reinforcement learning and recursive inference. *arXiv preprint arXiv:2605.30914*, 2026.
- Zichen Xie, Mrigank Pawagi, Yuxin Liu, Aaditi Rai, Lize Shao, John Berberian, Sicong Che, and Wenxi Wang. Vericontest: A competitive-programming benchmark for verifiable code generation. *arXiv preprint arXiv:2605.08553*, 2026.
- Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking verifiable code generation. *arXiv preprint arXiv:2505.23135*, 2025.
- Zhe Ye, Aidan Z. H. Yang, Huangyuan Su, Zhenyu Liao, Samuel Tenka, Zhizhen Qin, Udaya Ghai, Dawn Song, and Soonho Kong. Intent-aligned formal specification synthesis via traceable refinement. *arXiv preprint arXiv:2604.10392*, 2026.

Appendix A: Model routes and generation settings

All reported invocations occurred in September 2026. Runtime and inference-profile calls used Amazon Bedrock in

us-west-2; Terra used the Bedrock Mantle Responses route. Table 1 records the identifiers used in the reported campaigns and auxiliary cross-family diagnostics.

Optional decoding fields were not overridden. Claude Sonnet 5 and Opus 5 used adaptive thinking with the provider-default high effort and no temperature parameter. GPT-5.6 Terra used standard mode with the provider-default medium reasoning effort and no temperature parameter. DeepSeek V3.2, Devstral 2, Qwen3-Coder, GLM-5, and MiniMax M2.5 used their Runtime provider defaults for temperature, top- p , top- k , and stopping behavior. These defaults are identified by model and invocation month because provider defaults may evolve. Every method used three calls per episode; failed parses were retained as failures.

Appendix B: Prompt templates

The templates below use braces for task-specific fields. Line wrapping and punctuation are normalized for print; the serialized replay payloads belong in the anonymous supplement. INTENTSHIELD call 1, Model-Judgment call 1, and every PHASE call use the DIRECT template.

DIRECT system prompt.

You are an expert in formal verification and adversarial testing. Your task is to expose incomplete formal specifications by finding programs that satisfy a weakened formal contract while violating $\hookrightarrow$ the intended behavior described in natural language.

Your goal is NOT to implement the task correctly. Your goal is to $\hookrightarrow$ find a concrete example that exposes a gap between what the contract verifies and what the task actually requires.

DIRECT user prompt.

Task Description: {natural_language_task}

Weakened Verus Contract: {weakened_contract_code}

The contract has been deliberately weakened. Find a concise pure $\hookrightarrow$ Rust expression that:

1. Verifies successfully under this weakened contract.
2. Violates the intended behavior described in the task.

Known-Wrong Mutation Examples: {mutation_examples}

Use these as guidance for specification gaps to explore. Constraints:

- Return a pure Rust expression only.
- No statements, loops, unsafe blocks, I/O, or macros.
- Be deterministic and parse as valid Rust syntax.
- Prefer concise expressions.

Tracking Label: {replicate_label}
Respond with ONLY the candidate expression.

INTENTSHIELD calls 2 and 3.

Task Description: {natural_language_task}
Weakened Verus Contract: {weakened_contract_code}
Previous Candidate: {previous_expression}

Feedback from Previous Round:

- Rust Compilation: {rustc_status}
- Weakened Contract Verification: {verus_weakened_status}
- Visible Test Results: {visible_test_table}
- Pass: {num_pass}/{num_visible}
- Fail: {num_fail}/{num_visible}

Find a program that the weakened contract admits but the intended $\hookrightarrow$ task behavior rejects. If the candidate failed to compile or verify, $\hookrightarrow$ revise it to pass both gates. If it passes the contract but fails visible tests, consider preserving that failure mode.

Tracking Label: {replicate_label}
Respond with the revised candidate expression.

Model-Judgment calls 2 and 3.

Task Description: {natural_language_task}
Weakened Verus Contract: {weakened_contract_code}
Previous Candidate: {previous_expression}

Assess the previous candidate:

1. Does it verify under the weakened contract?
2. Does it violate the intended task behavior?
3. Is it a valid specification-completeness witness?

Provide the assessment, then revise the candidate if needed.
Tracking Label: {replicate_label}
Format:
Assessment:
Revised Candidate:

A2 specification repair.

Task Description: {natural_language_task}
Current Weakened Contract: {weakened_contract_code}
Certified Witness: {witness_expression}

Test Failure Evidence:
Input: {failing_test_input}
Expected Output: {expected_output}
Witness Output: {witness_actual_output}

Propose a minimal contract strengthening that rejects this witness $\hookrightarrow$ and similar wrong behaviors, preserves intended correct $\hookrightarrow$ implementations, and adds only necessary postconditions. Return the complete $\hookrightarrow$ repaired Verus function signature with ensures clauses.

A2 implementation synthesis.

Task Description: {natural_language_task}
Formal Verus Contract: {specification_code}
Official Test Cases (visible): {visible_test_cases}

Synthesize a pure Rust expression that satisfies the contract and passes all visible test cases. Return only the implementation expression.

Appendix C: Task-level Verus results

Appendix D: Downstream task results

Appendix E: Transfer and mechanism diagnostics

Appendix F: Representative certified witnesses

Appendix G: Reproducing the reported analysis

The public companion package contains the task-level CSV files and scripts used to recompute the reported intervals, tests, and figures. From the supplement root, run:

```
bash reproduce.sh
```

This regenerates the numerical report and every paper figure without making model calls.

The current package supports complete reproduction of the reported analysis from the retained task-level data. Full replay of the model-facing experiments will additionally require anonymized prompts, responses, checker outputs, request metadata, and runtime information.

Table 1: Model identifiers used in Paper 1. The first four models form the primary A1 matrix.

Model	Bedrock identifier	Use
Claude Sonnet 5	global.anthropic.claude-sonnet-5	A1, A2, Dafny, Lean
DeepSeek V3.2	deepseek.v3.2	A1 primary
Devstral 2 123B	mistral.devstral-2-123b	A1 primary
Qwen3-Coder 480B-A35B	qwen.qwen3-coder-480b-a35b-v1:0	A1, Dafny, Lean
Claude Opus 5	global.anthropic.claude-opus-5	cross-family diagnostic
GLM-5	zai.glm-5	cross-family diagnostic
MiniMax M2.5	minimax.minimax-m2.5	cross-family diagnostic
GPT-5.6 Terra	openai.gpt-5.6-terra	A2 and Dafny

Table 2: Complete four-method campaign and omitted properties. Each policy contributes 12 outcomes per task. Primary inference compares Direct and INTENTSHIELD; Model-Judgment is secondary.

Task	Stratum	Omission	Direct	INTENTSHIELD	Model-J.	PHASE	D-IS
cf124A	Residual	Equality	9/12	3/12	4/12	9/12	+50.0
cf492A	Residual	Maximality	8/12	3/12	3/12	8/12	+41.7
cf92A	Residual	Equality	7/12	2/12	2/12	7/12	+41.7
cf271A	Residual	Minimality	6/12	2/12	3/12	6/12	+33.3
cf519C	Residual	Maximality	5/12	2/12	2/12	5/12	+25.0
cf4A	Total	Output relation	11/12	7/12	8/12	11/12	+33.3
cf110A	Total	Output relation	10/12	6/12	5/12	10/12	+33.3
cf617A	Total	Output relation	9/12	5/12	4/12	9/12	+33.3
cf50A	Total	Output relation	2/12	4/12	1/12	2/12	-16.7
cf47A	Total	Output relation	6/12	1/12	1/12	6/12	+41.7
cf318A	Total	Output relation	5/12	1/12	0/12	5/12	+33.3
cf486A	Total	Output relation	7/12	3/12	1/12	7/12	+33.3
Total			85/144	39/144	34/144	85/144	+31.9

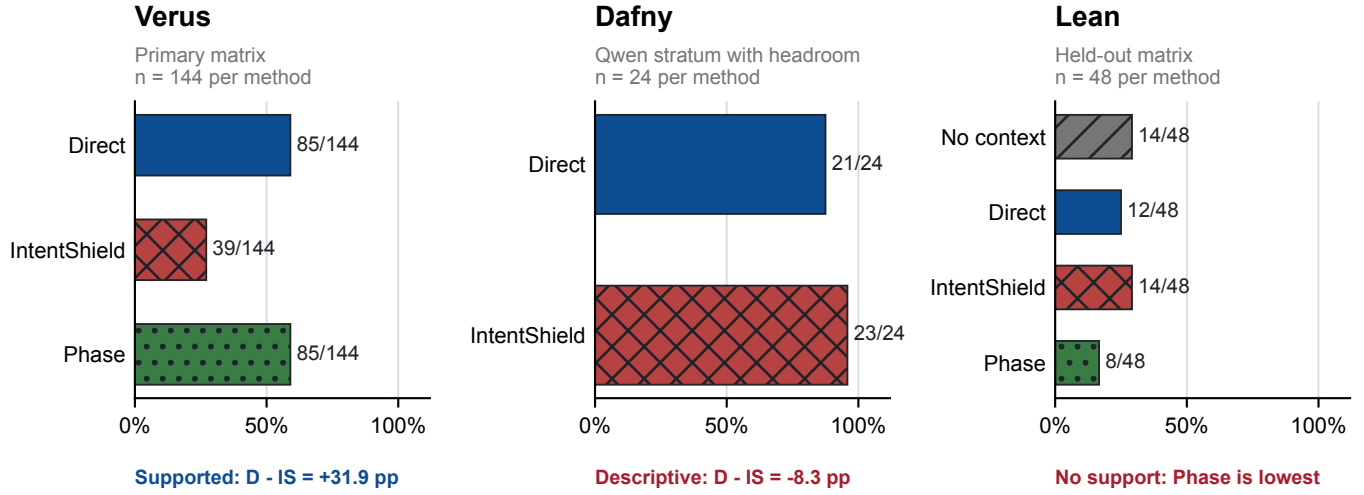
Table 3: Successful implementation attempts out of three. The last four tasks were not part of the primary discovery matrix.

Task	Original	IS-Rev.	Phase-Rev.	Task	Original	IS-Rev.	Phase-Rev.
cf124A	0/3	2/3	3/3	cf50A	3/3	3/3	3/3
cf492A	0/3	2/3	3/3	cf47A	1/3	2/3	2/3
cf92A	0/3	1/3	2/3	cf318A	0/3	2/3	3/3
cf271A	1/3	2/3	3/3	cf486A	0/3	1/3	2/3
cf519C	0/3	2/3	3/3	cf200A	1/3	2/3	2/3
cf4A	3/3	3/3	3/3	cf228A	0/3	2/3	2/3
cf110A	1/3	2/3	2/3	cf263A	1/3	1/3	2/3
cf617A	2/3	2/3	2/3	cf337A	1/3	1/3	2/3
Total				14/48	30/48	39/48	

Table 4: Examples of programs admitted by a weakened contract but rejected by the restored obligation and an official case.

Task	Omitted property	Candidate	Official disagreement
cf124A	Functional equality	n	Input 3 1 1; expected 2, produced 3.
cf492A	Maximality	0	Input 1; expected 1, produced 0.
cf271A	Minimality	9876	Input 1987; expected 2013, produced 9876.
cf4A	Output relation	w >= 6	Input 4; expected YES, produced NO.
cf110A	Output relation	true	Input 4; expected NO, produced YES.

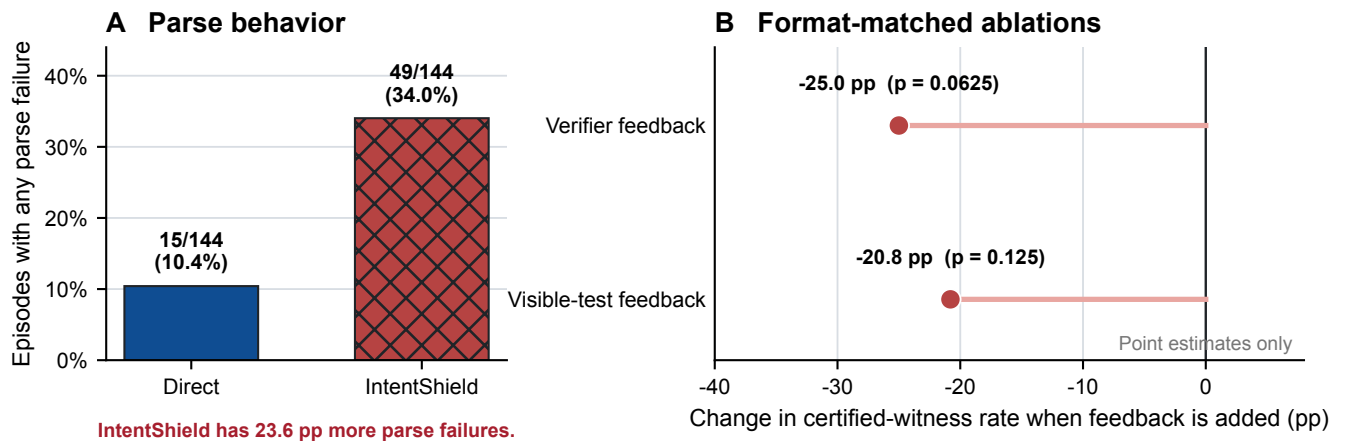
The Verus result does not reproduce across proof substrates



Panels use different task matrices and are compared for direction and scope, not absolute difficulty.

Figure 4: The primary Verus direction does not reproduce in the available Dafny and Lean evaluations. The task matrices differ and should not be compared as measures of absolute difficulty.

Mechanism diagnostics are descriptive, not causal



After restricting to parse-clean episodes, Direct retains a 23.6 pp certified-witness advantage. This is descriptive because parse success is post-treatment.

Figure 5: Descriptive mechanism checks. Parse success is post-treatment, and the format-matched ablations are underpowered point estimates rather than isolated causal effects.